ProofPay: Agent Specification

Scope: the four agents (Scoping, Verifier, Recovery, Dispute), what each takes in, what it does, what it calls, and how it plugs into the rest of the system. This document comes before the implementation plan so the team can settle the open design decisions in section 8 first.

Core rule for every agent: **the LLM proposes, the policy engine decides, code executes.** No agent calls PayPal directly.

0. Shared Agent Contract

All four agents follow one runtime pattern, so they can be built, tested and integrated the same way.

0.1 Runtime pipeline

```
trigger (event / user action / scheduler tick)
  -> load context (DB reads only)
  -> deterministic code steps (validators, collectors, rules)
  -> optional LLM call (schema-constrained JSON, budgeted)
  -> Proposal (structured action)
  -> Policy engine (mandate guard) -> allow | deny | needs_approval
  -> Executor (PayPal / email / DB) with idempotency key
  -> Observe (webhook or simulator) -> state transition
  -> Audit row at every step
```

0.2 Proposal object (what every agent returns)

```
{
  "agent": "verifier",
  "subject": {"type": "milestone", "id": "m_102"},
  "action": "release_milestone_payout",
  "params": {"amount": 150.00, "currency": "USD", "recipient_id": "u_7"},
  "reason": "All 4 hard criteria passed, score 1.00",
  "inputs_used": ["submission:s_55", "criteria:plan_hash_ab12"],
  "llm_used": true,
  "confidence": 0.95
}
```

0.3 Policy decision object (what the policy engine returns)

```
{"verdict": "allow | deny | needs_approval",
 "checks": [{"rule": "max_release_amount", "passed": true},
            {"rule": "no_open_dispute", "passed": true}],
 "blocked_by": null}
```

A `deny` is logged and escalated to a human. It is never bypassed or retried with different wording.

0.4 Rules common to all agents

Rule	Detail
LLM wrapper	One `llm_client.call(task, json_schema, budget_key)` function. It enforces call budgets, validates the JSON schema, retries once on invalid output, then falls back to the rule path. Provider sits behind an interface so a free-tier model can be swapped in (zero-cost constraint).
LLM budget	Max 2 calls per case or milestone, plus a daily cap. Counter lives in the DB and is checked by the policy engine.
Untrusted input	Evidence content, customer replies and job text are data, never instructions. They are passed in clearly delimited fields and never concatenated into the system prompt.
Idempotency	Every webhook is deduped by event ID. Every PayPal write carries an idempotency key (for example `PayPal-Request-Id`, or a payout `sender_batch_id` built from subject ID + attempt).
Audit	One append-only row per step: timestamp, subject, actor (agent, policy, executor, human), decision, one-line reason, inputs used, checks passed, PayPal ID, outcome.
Fallback	If the LLM is empty, invalid or over budget, the agent falls back to the rule table or a template and logs the fallback.

Rule	Detail
Terminal states	Final. A human action is required to reopen.
Read vs write tools	Read tools are ungated. Write tools always pass through the policy engine.

1. Scoping Agent (Escrow)

1.1 Purpose

Turn a plain-text job description into a milestone plan with amounts, deadlines and **machine-checkable** acceptance criteria. Both sides approve before anything is funded.

1.2 Trigger

Client submits the job intake form (Next.js). Also re-runs validators (not the LLM) when either side edits the plan.

1.3 Inputs

Input	Source
Job description (text)	Intake form
Total budget and currency	Intake form
Overall deadline	Intake form (optional)
Client and freelancer IDs	DB
Criteria library (allowed check types)	Config
Policy limits (max project total, max per release, min milestone amount)	`policy_config`

1.4 Criteria library (the key design constraint)

The LLM may only pick from a fixed library. It cannot invent free-form criteria. This is what makes the Verifier reliable.

Type	Check	Params example
`url_returns_200`	HTTP GET succeeds	`{url_ref: "deployed_url"}`
`url_contains_text`	Response contains strings	`{strings: ["Pricing", "Contact"]}`
`github_pr_merged`	PR state is merged	`{repo, pr_ref}`
`github_checks_passing`	CI status success	`{repo, ref}`
`repo_file_exists`	Path present in repo	`{path: "README.md"}`
`file_type_size`	Extension and size limit	`{ext: "pdf", max_mb: 10}`
`file_has_sections`	Headings present	`{headings: ["Summary", "Budget"]}`
`manual_review`	Soft criterion, shown to client, not scored	`{prompt: "Logo matches brand"}`

Each criterion: `{id, type, params, weight, must_pass}`. Rule: every milestone needs at least one hard (non-manual) criterion, and hard criteria carry at least 70% of the weight.

1.5 Processing steps

1. **Pre-validate (code).** Length limits, currency supported, budget within policy limits. Reject early with a clear message.
2. **LLM call 1.** Decompose into 2 to 6 milestones. Output is schema-constrained JSON: title, amount, deadline offset, criteria list from the library.
3. **Validators (code).** Amounts sum to the total (fix rounding on the last milestone). Deadlines ordered and within the project deadline. Every milestone has at least one hard criterion; all types are in the library; params are complete. Each amount is at or above the minimum and at or below the per-release cap.
4. **Repair (LLM call 2, only if validation fails).** Send the exact error list back. If it still fails, fall back to a template plan (single-milestone or split-by-thirds) and flag it for human edit.
5. **Clarification path.** If the job text is too vague, return `needs_clarification` with at most 3 questions, once. No second round.
6. **Persist as DRAFT** and show the plan in the UI.

7. **Approval.** Client and freelancer can edit. Every edit re-runs the validators with no LLM call. When both approve, the plan is locked: state becomes `PLAN_APPROVED` and a `plan_hash` (SHA-256 of milestones and criteria) is stored.

1.6 Outputs

Output	Where it goes
`projects`, `milestones` rows with criteria JSON	Postgres
`plan_hash`	Stored on the project, referenced by the Verifier
Audit rows	Audit log
`plan.approved` event	Triggers the funding step in the UI

1.7 Integrations

Direction	Component	How
Upstream	Next.js intake form	`POST /projects`
Upstream	Policy config	Read limits
Downstream	Funding	Client pays the total via PayPal Orders v2 after `PLAN_APPROVED`
Downstream	Verifier	Reads locked criteria by `plan_hash`
Downstream	AG Grid ledger	Master-detail rows: project, milestones, criteria
Downstream	Reputation	None at this stage

1.8 Failure handling

Failure	Handling
LLM invalid or empty JSON	Repair call, then template fallback, log it

Failure	Handling
Amounts do not sum	Code corrects the last milestone or rejects with a message
Vague job text	One clarification round
Edit breaks validators	Block the save, show which rule failed

1.9 Acceptance tests

- Golden set of 10 job descriptions: every output passes all validators.
- Property tests: random edits never produce a plan whose amounts do not sum.
- A plan cannot be funded unless both approvals exist and `plan_hash` matches.

2. Verifier Agent (Escrow)

2.1 Purpose

Check submitted work against the locked criteria. **Code checks the facts; the LLM only writes the rationale.** The score decides the route.

2.2 Trigger

Freelancer submits evidence for a `FUNDED` milestone. Resubmissions are allowed up to 2 attempts per milestone (configurable).

2.3 Inputs

Input	Source
Milestone and locked criteria	DB via `plan_hash`
Evidence refs: GitHub PR URL, deployed URL, uploaded file	Submission form
Submission history	DB
Thresholds and policy limits	`policy_config`

2.4 Evidence collectors (code, deterministic)

Each collector returns facts of the form `{criterion_id, passed, value, source, checked_at}`.

Collector	Does	Safety
GitHub collector	PR state, merged flag, CI check status, files changed	Read-only token; rate-limit backoff
HTTP prober	Status code, text contains, response time	Allowlist schemes, block private and loopback IPs (SSRF), timeout, 3 probes spread over time with majority vote
File inspector	Type, size, hash, headings present	Size cap, no execution of uploaded content, scan only extracted text

2.5 Scoring and routing (code)

```
score = sum(weight_i for hard criteria passed) / sum(weight_i for all hard criteria)
```

Condition	Route	Next state
Score at or above 0.90 and every `must_pass` criterion passed	Auto-release proposal	`RELEASE_PENDING`
Score 0.60 to 0.89, or any `must_pass` failed with the rest good, or manual-review criteria present	Client review with rationale attached	`IN_REVIEW`
Score below 0.60	Back to freelancer with the failed list. After the attempt limit, route to the dispute path	`SUBMITTED` (retry) then `DISPUTED`

Manual-review (soft) criteria never change the score. They are listed for the client to confirm. Thresholds are config, not hardcoded.

2.6 The single LLM call

Input: the structured facts plus criteria titles. Output: a rationale of at most 150 words plus one plain-language line per criterion. Post-checks in code: the rationale may only reference known criterion IDs, and pass/fail shown in the UI always comes from the facts, never from LLM text. One call per submission; on failure, a template rationale is generated from the facts.

2.7 Release path (when the route is auto-release or client approves)

1. Verifier emits a `release_milestone_payout` proposal.
2. Policy engine checks: amount within per-release limit; above the auto-action limit means `needs_approval`; recipient account verified; milestone funded and ledger balance sufficient; no open dispute; no earlier release for this milestone (duplicate guard).
3. Reconciliation gate confirms the funding capture exists in PayPal records.
4. Executor calls PayPal Payouts through the MCP server / Agent Toolkit with an idempotency key.
5. Webhook `PAYMENT.PAYOUTS-ITEM.SUCCEEDED` (confirm exact name in PayPal docs) moves the milestone to `RELEASED`, updates the ledger and writes the reputation event.

2.8 Outputs

Output	Where
`submissions` row: facts, score, rationale, route	Postgres
`release_milestone_payout` proposal	Policy engine
Client review task with rationale	Dashboard
Ledger entry and reputation event on release	Postgres
Audit rows	Audit log

2.9 Integrations

Direction	Component	How
Upstream	Scoping agent	Locked criteria and `plan_hash`

Direction	Component	How
Upstream	Submission UI	`POST /milestones/{id}/submissions`
Downstream	Policy engine and executor	Release proposal
Downstream	PayPal Payouts via MCP / Toolkit	Payout call
Downstream	Dispute agent	Low score after final attempt, or client rejection
Downstream	AG Grid ledger	Row updates through webhook-driven refresh
Sponsor	AG Grid	Submission detail in master-detail row

2.10 Failure handling

Failure	Handling
GitHub rate limit or outage	Retry with backoff; mark `VERIFY_DELAYED`, never a fail
URL flaky	3 probes over time, majority vote
Evidence unreachable or empty	Treat that criterion as failed with reason "unreachable"
Payout API error	Retry with backoff, then `ACTION_FAILED` and escalate
Payouts not enabled in sandbox	Fall back to Orders and Invoicing for the release step
Amount mismatch at release	Stop and flag for review

2.11 Acceptance tests

- Seeded submissions with planted labels (clear pass, clear fail, borderline). Verifier route matches the label.
- Rationale never contradicts the fact table (automated check).
- Duplicate submission event or duplicate webhook produces exactly one payout.

- Over-limit release is blocked by the mandate guard on camera.

3. Recovery Agent (Recovery)

3.1 Purpose

Recover money from failed subscription payments and overdue invoices without contacting people who already paid, and stop at the right time. Rules first; the LLM is limited to 2 calls per case.

3.2 Triggers

Trigger	Source
Failed subscription payment	PayPal webhook (`BILLING.SUBSCRIPTION.PAYMENT.FAILED`, confirm name) or simulated event
Overdue invoice	Scheduled scan of invoices (and PayPal invoice events where available)
Escrow funding or instalment failure	Internal event from the Escrow engine
Next ladder step due, promise-to-pay date	Scheduler tick
Customer reply or payment confirmation	Webhook or simulator

3.3 Inputs

Input	Source
Case: type, amount, due date, failure code	Event and DB
Customer: language, timezone, opt-out flag, history	`customers`
Prior attempts and results	`attempts`
Reconciliation matches	Reconciliation service

Input	Source
Policy config: contact cap, quiet hours, gap, amount limits	`policy_config`

3.4 Processing loop (per case, per trigger)

1. **Create or load the case** (`OPEN`). Dedupe by event ID.
2. **Gate 1, already paid?** `check_already_paid` runs the reconciliation match across PayPal, merchant sales and bank records. Auto-match: close as `PAID_ELSEWHERE`, zero contact, terminal. Human-review tier: hold the case, no contact, create a review task. No match: continue.
3. **Diagnose.** Rules map failure code and invoice status to a diagnosis: insufficient funds, expired card, temporary hold, forgot to pay, large overdue invoice, hard decline. Only messy free text (for example a customer reply such as "I'll pay Friday") uses **LLM call 1**, with schema-constrained output: diagnosis, promised date if any.
4. **Prioritize.** `priority = amount x recovery likelihood` from a rule table. The daily action queue is sorted by it.
5. **Select strategy.** Table lookup (below), then pick a template by diagnosis and language, and choose timing (payday alignment, customer-timezone quiet hours). **LLM call 2** is optional, used only if no template fits.
6. **Propose, policy check, execute.** The proposal goes through the mandate guard, then the executor.
7. **Observe and stop check.** After each action, wait for the webhook or simulator result. Stop on the hard stops below, otherwise schedule the next ladder step.

3.5 Diagnosis to action table

Diagnosis	First action	Escalation ladder
Insufficient funds	Retry at payday-aligned time plus short reminder	Payment link, then split offer
Expired card	Payment-method update request	Payment link, then human
Temporary hold	Wait, retry once	Reminder, then payment link
Forgot to pay	Friendly reminder	Payment link, then split offer

Diagnosis	First action	Escalation ladder
Large overdue invoice	Offer split or Pay Later	Payment link, then human
"I'll pay Friday"	Log promise, remind the day before	If missed, resume ladder at next step
Suspicious hard decline	No retry	Escalate to human immediately

3.6 Hard stops (case state machine)

```
OPEN -> RECONCILING -> DIAGNOSED -> ACTION_PENDING_APPROVAL (if large)
     -> ACTION_SENT -> AWAITING_RESULT
     -> RECOVERED | HALTED | UNRECOVERED | PAID_ELSEWHERE   (terminal)
```

Stop	Result
Payment received	`RECOVERED`, amount added to the headline number
Dispute opened	`HALTED`, zero further contacts
Opt-out	`HALTED`
Contact cap reached (4)	`UNRECOVERED` with reason
Paid elsewhere	`PAID_ELSEWHERE`

3.7 Policy checks before every write

Contact count under 4, minimum 48 hours since the last contact, outside quiet hours (9 pm to 8 am customer time), no dispute, no opt-out, amount under the auto-action limit (above it, human approval), daily action cap, LLM budget, channel in the allowed list (email and payment link only).

3.8 Tools

Tool	Type	Gated
`get_case`, `get_customer_history`	Read	No

Tool	Type	Gated
`check_already_paid`	Read	No
`schedule_retry`	Write	Yes
`send_invoice_reminder`	Write	Yes
`create_payment_link`	Write	Yes
`offer_split_payment`	Write	Yes
`log_promise_to_pay`	Write	Yes
`escalate_to_human`	Write	No

3.9 Outputs

Output	Where
`attempts` rows, case status, diagnosis, priority	Postgres
Outreach (email with template, payment link)	Executor
Recovered $ and unrecovered reasons	Dashboard headline and exception list
Audit rows	Audit log

3.10 Integrations

Direction	Component	How
Upstream	Webhook receiver and scheduler	Events and ticks
Upstream	Reconciliation gate	Already-paid check, mandatory first step
Downstream	Policy engine and executor	All writes
Downstream	PayPal Subscriptions, Invoicing, Orders, Pay Later	Retry or capture outstanding balance, reminder, payment link, split offer
Downstream	Email (free-tier provider or SMTP)	Template-based outreach

Direction	Component	How
Downstream	Dispute agent	A dispute event halts the case immediately
Demo mode	Customer simulator	Replies according to hidden profiles
Evaluation	Harness	Same runner with `strategy=baseline` (retry 3 times, same email to all)
Sponsors	AG Grid, Elastic (optional), Zapier (optional)	Case grid and unrecovered view, audit search, human-approval step

3.11 Failure handling

Failure	Handling
Duplicate webhook	Dedupe by event ID, process once
PayPal API error	Retry with backoff, then `ACTION_FAILED` and escalate
LLM bad output or over budget	Fall back to rule table, log it
Policy block	Log, escalate to a human, never bypass
Amount mismatch on retry	Stop and flag for review

3.12 Acceptance tests

- Over 150 seeded cases: zero contacts to "already paid" customers.
- A dispute event mid-ladder halts the case and no further action is proposed or executed.
- No case exceeds 4 contacts, and no two contacts are under 48 hours apart.
- Share of cases handled with no LLM call is reported.
- Agent vs baseline run on identical cases produces the comparison table.